DEPARTMENT OF APPLIED MATHEMATICS
M.SC. DATA SCIENCE

SUBJECT : BIG DATA ANALYSIS

Submitted By:
Adarsh Jha (25-60-08)
Shivam (25-60-05)

Submitted To : Dr. Yogeshwar Singh Dadhwal

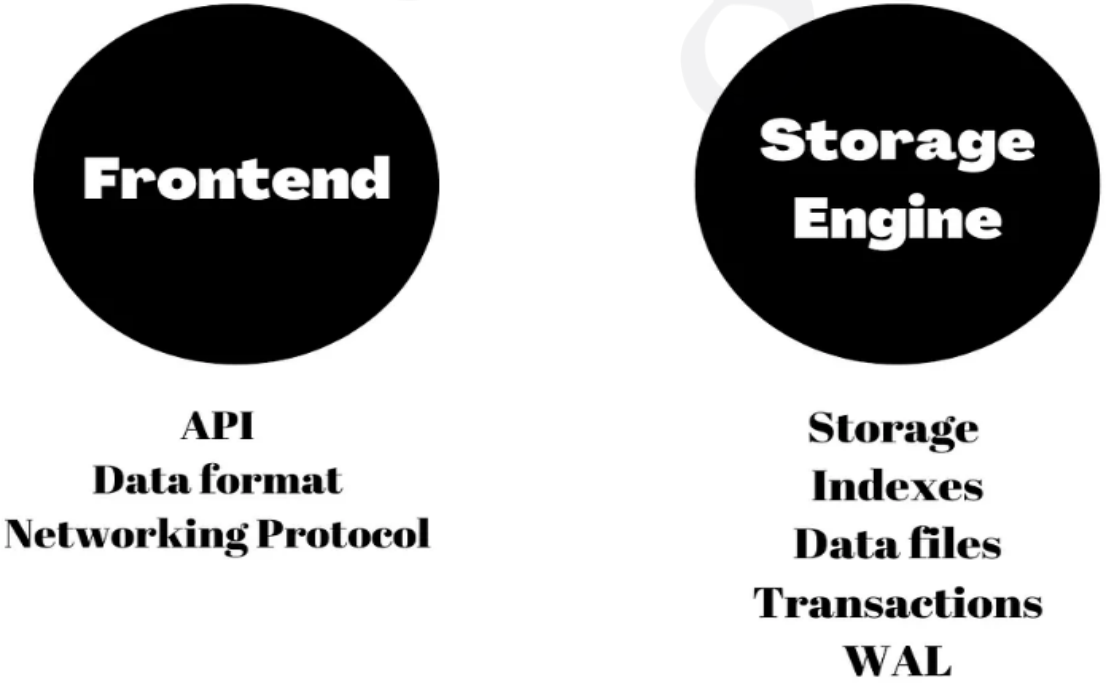
mongoDB

CONTENT :

- Introduction
- CRUD Operation
- Arrays Functions
- Count, Sort
- Limit, Skip
- Aggregation
- Map Reduce
- Mongo import and export

DATABASE

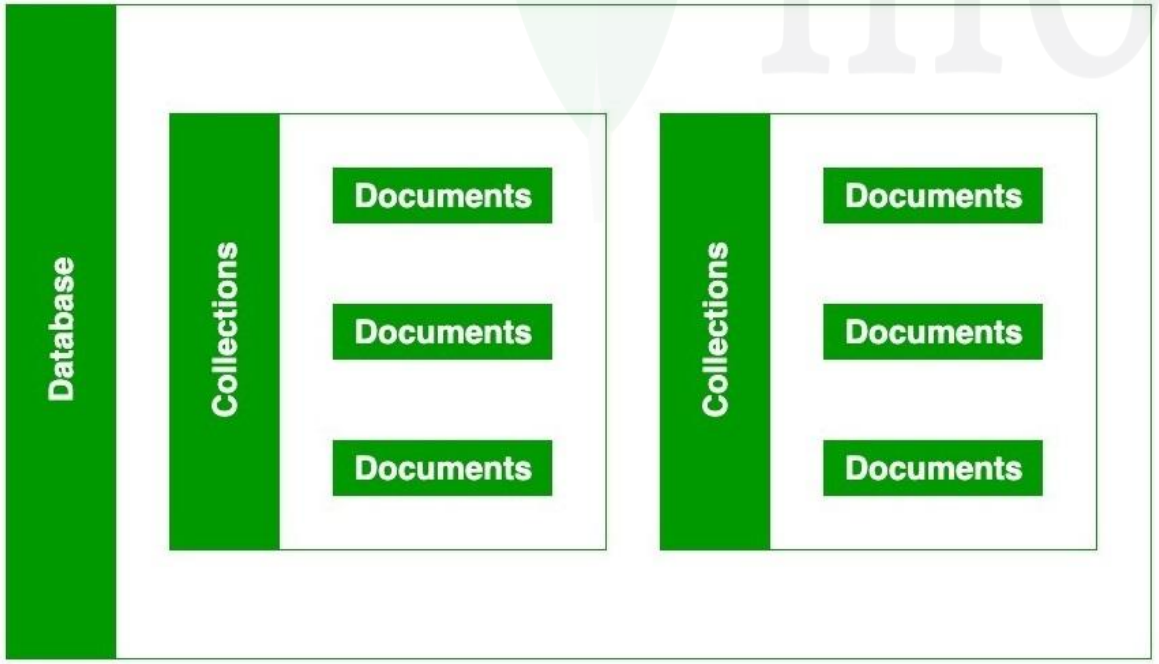
- A database to me is two parts.
- A frontend and a backend.
- The frontend is the API (SQL vs get/set) & data format (tables vs documents).
- The backend is the storage engine defines how data is stored on disk, indexes, WAL etc.
- The core difference between SQL/NOSQL is the frontend.

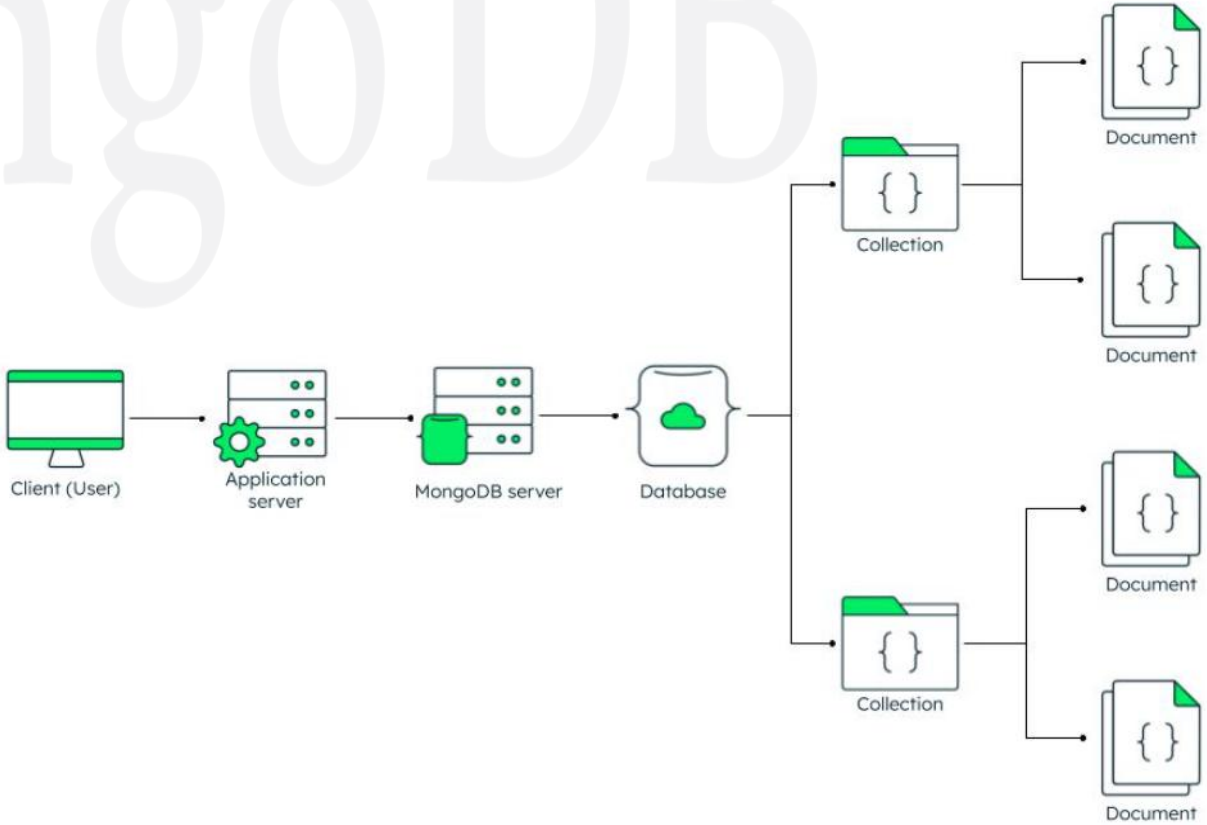


What is MongoDB?

MongoDB is a NoSQL, document-oriented database.
The Intuition: Instead of thinking in tables and rows, think in folders and files.

- **Database** is the filing cabinet (a.k.a. “Container”)
- **Collection** (like SQL table) is a folder inside that cabinet.
- **Document** (like SQL row) is a specific file inside that folder.
- **Fields** (like SQL columns) is a “key” which represents individual piece of information in a document.





Why MongoDB ?

- Since MongoDB allows storing nested data within a single document, no need of complex and expensive JOINS. This **improves READ performance**.

- Since, MongoDB uses JSON/BSON format, it makes data retrieval and storage **highly intuitive**; because it closely resembles application objects (or classes) in languages like JavaScript, Python, PHP.

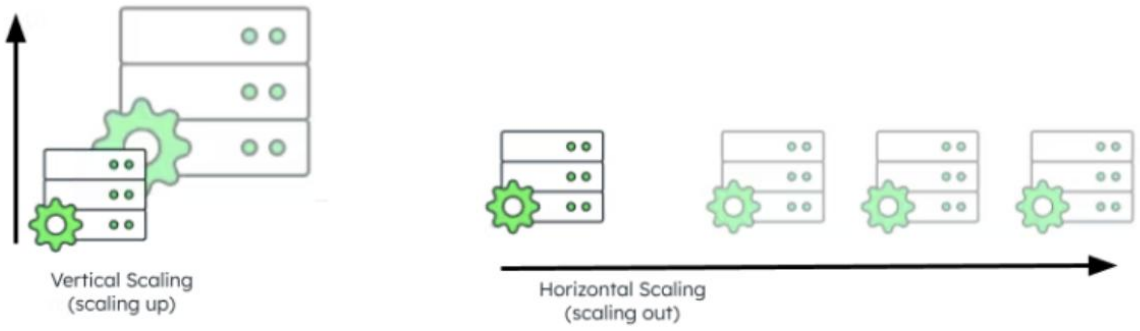
- Due to JSON/BSON format, ORM overhead (translating code objects into table rows) is eliminated. This results in **fast dev cycle**.

- Since, MongoDB doesn't have a rigid schema it allows **adding/changing fields on-the-fly**. This results in **faster prototyping** without expensive need of migrations.

- Lack of rigid structure also helps in **handling real-time high throughput data** (e.g. IoT)

- MongoDB is built from the ground-up to use horizontal scaling a.k.a **"Sharding"**.

- **No ACID compliance**, **No fixed Schema**, **Flexible Data Models -** all these contribute towards distributed computing friendly architectural design.

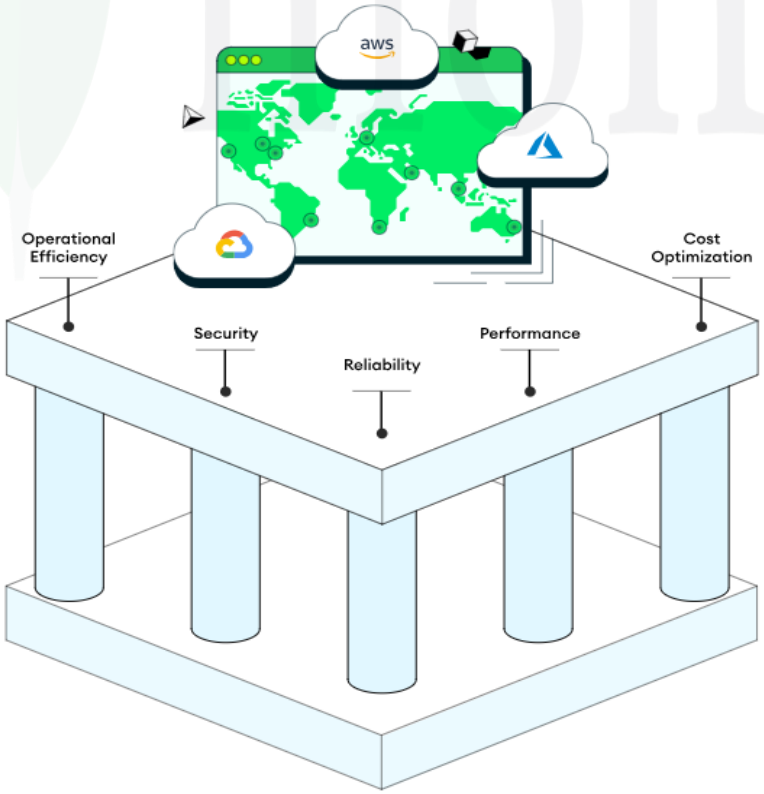


Setup and Installation

1.MongoDB Atlas (Cloud): The easiest way. It's a fully managed cloud database. You sign up, click a few buttons, and you have a free MongoDB cluster running without installing any database software locally.

2.Local Installation (Community Server): Downloading the MongoDB Community Server directly to your OS (Windows, macOS, Linux). Good for completely offline development.

3.Docker Container: For developers who want isolated environments. Running "`docker run -d -p 27017:27017 --name mongo-db mongo`" spins up a database instantly.



Install MongoDB Community Edition

Copy page

Select your operating system to view installation instructions for MongoDB Community Edition. On certain operating systems, you can include or exclude MongoDB Search using the corresponding selector.

OPERATING SYSTEM: Windows

INSTALLATION METHOD: Zip File

Install MongoDB Community on Windows using the Zip File

Overview

Use this tutorial to install MongoDB Community on Windows by downloading and extracting the zip file.

MongoDB Version

This tutorial installs MongoDB 8.2 Community Edition. To install a different version of MongoDB Community, use the version drop-down under **Self-Managed Deployments** on the left side of this page to select the documentation for that version.

Installation Method

This tutorial installs MongoDB on Windows by extracting from a zip file. You can also install MongoDB on Windows by these other methods:

- Install MongoDB using msiexec.exe
- Install MongoDB using the MSI Installer

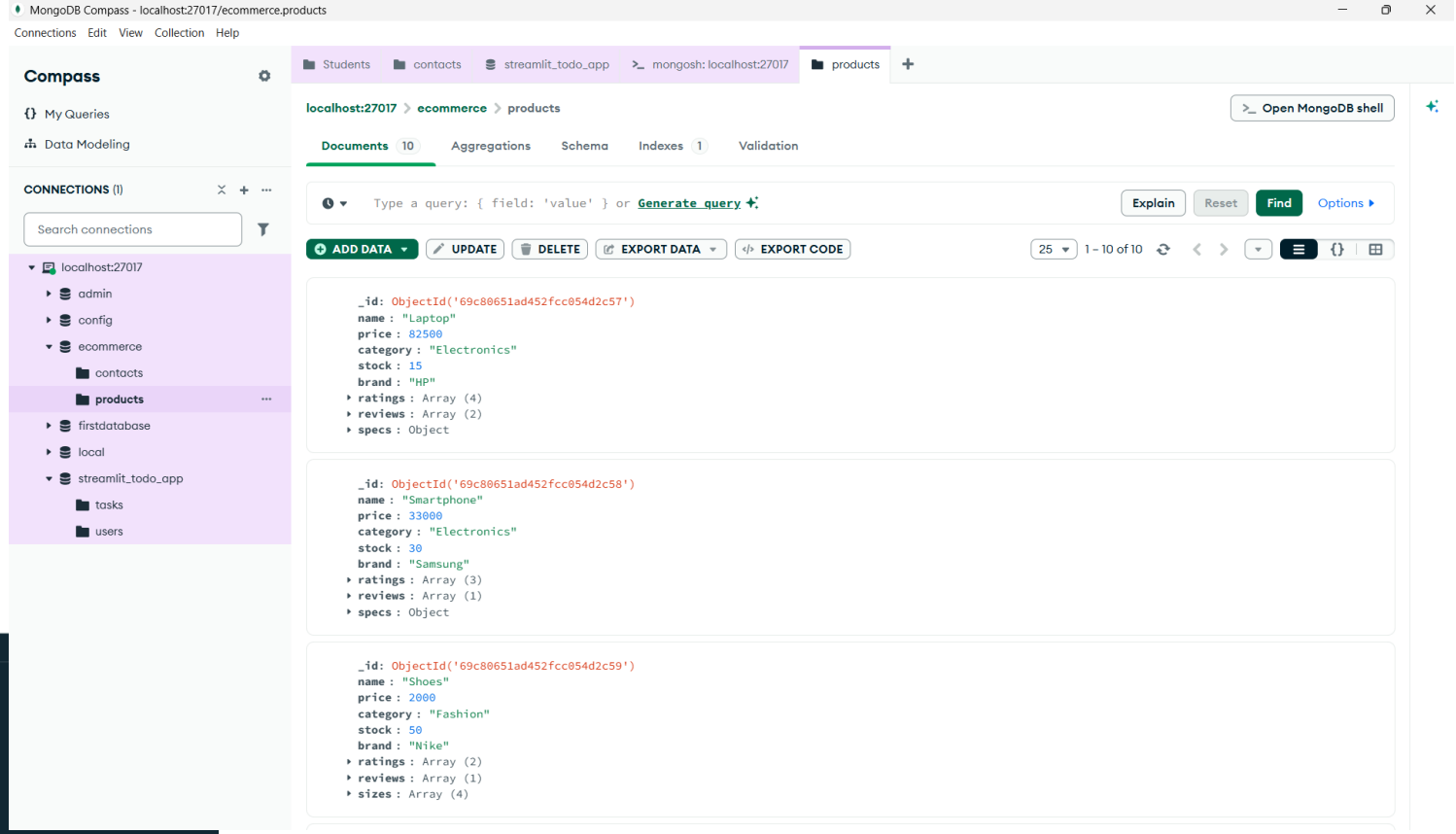
mongoDB

on

docker

MongoDB Compass vs. MongoDB CLI

MongoDB Compass: The official graphical user interface (GUI). It allows you to visually explore your data, click through collections, build queries with a point-and-click interface, and visually construct complex aggregation pipelines. It is excellent for data exploration.




```
>_MONGOSH
> db["products"].find()
<
> show databases
< admin                40.00 KiB
  config               36.00 KiB
  ecommerce           136.00 KiB
  firstdatabase         8.00 KiB
  local                64.00 KiB
  streamlit_todo_app  132.00 KiB
> use ecommerce
< switched to db ecommerce
> show collections
< contacts
  products
> db.products.find()
< {
    _id: ObjectId('69c80651ad452fcc054d2c57'),
    name: 'Laptop',
    price: 82500,
    category: 'Electronics',
    stock: 15,
    brand: 'HP',
    ratings: [
      4,
      5,
      4,
      2
    ],
    reviews: [
      {
        user: 'Amit',
        comment: 'Good performance',
        rating: 4
      },
```

MongoDB CLI (`mongosh`): The MongoDB Shell. It is a fully functional JavaScript and Node.js environment directly in your terminal. It is used for quick administrative tasks, scripting, and executing raw commands.

Hands-On Demonstration: CRUD Operations

Create

- **CLI:**

```
db.users.insertOne({ name: "Alice", age: 28, skills: ["Python"] });
db.users.insertMany([{ name: "Bob", age: 35 }, { name: "Charlie", age: 22 }]);
```

- **Compass:** Click the "ADD DATA" button -> "Insert Document". A JSON editor opens where you can type `{ "name": "Alice", "age": 28, "skills": ["Python"] }` and click Insert.

Read

- **CLI:** `db.users.find({ age: { $gt: 25 } });`

- **Compass:** In the "Filter" bar, type `{ age: { $gt: 25 } }` and hit "Find".

Update

- **CLI:** `db.users.updateOne({ name: "Alice" }, { $set: { age: 29 } });`

- **Compass:** Hover over a document, click the pencil icon (Edit), change the value of `age` to 29, and click "Update".

Delete

- **CLI:** `db.users.deleteOne({ name: "Charlie" });`

- **Compass:** Hover over the document and click the trash can icon (Delete), then confirm.

Arrays: Concepts

- In SQL, if a user has multiple skills, you would have to create a completely separate `user_skills` table and perform a **JOIN** operation to connect them.
- In MongoDB, you just embed the list directly in the user's document.
- MongoDB Arrays preserves the order of elements as they are stored.
- A single array can contain different data types, such as strings, numbers, booleans, or even other embedded documents and arrays.
- Flexibility: Different documents in the same collection can have different array structures.

What are Array Functions?

In MongoDB, array functions are generally categorized into –

- **Update Operators:** These operators modify the content of an array within a document.
- **Query Operators:** These are used in the `find()` method to locate documents based on array content.
- **Aggregation Expression Operators:**
  - They are specialized functions or operators used to calculate values and transform data.
  - These operators are grouped by the type of data or operation they handle.
  - Expressions are typically written as objects where the key is the operator (prefixed with $) and the value is the input or an array of inputs

UPDATE OPERATORS:

- **$push:** Appends a specified value to an array.
  - `db.products.updateOne({ name: "Laptop" },{ $push: { ratings: 5 } })`

- **$addToSet**: Adds a value to an array only if the value does not already exist, ensuring uniqueness.
  - `db.products.updateOne({ name: "Smartphone" },{ $addToSet: { ratings:5} })`

- **$pop:** Removes the first (using -1) or last (using 1) element of an array.
  - `db.products.updateOne({ name: "Laptop" },{ $pop: { ratings: 1 } })`

- **$pull**: Removes all array elements that match a specified condition.
  - `db.products.updateOne({ name: "Laptop" },{ $pull: { ratings: 4 } })`

- **$pullAll**: Removes all instances of the specified values from an array.
  - `db.products.updateOne({ name: "Laptop" },{ $pullAll: { ratings: [4, 5] } })`

QUERY OPERATORS

- **<u>$all</u>:** Matches arrays that contain every element specified in the query.
  - `db.products.find({ratings: { $all: [4, 5] }})`

- **<u>$elemMatch</u>**: Selects documents if at least one element in the array matches all specified criteria.
  - `db.products.find({reviews: {$elemMatch: {user: "Ravi", rating: 5}}})`

- **<u>$size</u>**: Selects documents where the array field is a specific length.
  - `db.products.find({ratings: { $size: 3 }})`

- **$exists:** These are used specifically to *filter* documents.
  - `db.products.find({ sizes: { $exists: true }})`

AGGREGATION EXPRESSION OPERATORS:

•**Arithmetic Operators:** Perform mathematical calculations like addition, subtraction, and multiplication (e.g., `$add`, `$subtract`, `$multiply`).

•**Comparison Operators:** Compare values and return a boolean (e.g., `$eq` for equals, `$gt` for greater than, `$lt` for less than).

```
db.Students.aggregate([{$project:{name:1,totalMarks:{$add:["$marks", 10]} } } ])
```

•**String Operators:** Manipulate or analyze text (e.g., `$concat`, `$toLower`, `$substr`, `$strLenCP`).

```
db.Students.aggregate([{$project:{fullName: { $concat:["Student: ","$name"]},
lowerName: { $toLower: "$name" }   } } ])
```

•**Array Operators:** Work with array fields (e.g., `$reduce`, `$filter` to select elements, `$in` to check for existence, `$size`, `$arrayElemAt, $concatArrays, $indexOfArray`).

•**Accumulators:** Used primarily in the `$group` stage to compute totals or averages across multiple documents (e.g., `$sum, $avg, $min, $max`).

•**Type Operators:** Check or convert BSON data types (e.g., `$type, $convert, $toInt`).

- **`$concat`** (Joins multiple strings together)- (Prepends "Student: " to the student's name)
- **$toLower** (Converts a string to lowercase)
- **$substr** (Returns a substring)
- **$strLenCP** (Returns the length of a string)
- **$reduce** (Applies an expression to each element to reduce them to a single value)
- **$filter** (Returns a subset of the array based on a condition)
- **$in** (Checks if a specified value exists in an array)
- **$size** (Returns the number of elements in an array)
- **$arrayElemAt** (Returns the element at a specific index)
- **$concatArrays** (Combines multiple arrays into one)
- **$indexOfArray** (Returns the index position of a specific value in an array)
- **$type** (Returns the BSON data type of a field)
- **$convert** (Converts a value to a specified data type)
- **$toInt** (Converts a value to an integer)

QUERY MODIFIERS:

- **count() / countDocuments():** Counts number of documents matching a query.
  - `db.products.countDocuments({ category: "Electronics" })`

- **sort():** Sorts the result set.
  - `db.products.find().sort({ price: 1 })  // Ascending`
  - `db.products.find().sort({ price: -1 })  // Descending`

- **limit():** Limits number of documents returned.
  - `db.products.find().limit(2)`

- **skip():** Skips a number of documents.
  - `db.products.find().skip(1)`

AGGREGATION:

- **Aggregation** is a way to **process and transform data** to get meaningful results—similar to SQL's GROUP BY, SUM, AVG, etc.

- It uses something called an **Aggregation Pipeline**, where data passes through multiple stages.

- Each stage modifies the data step by step. [Collection → Stage 1 → Stage 2 → Stage 3 → Result]

```
db.collection.aggregate([
  { stage1 },
  { stage2 },
  { stage3 }])
```

– Common Aggregation Stages:

Stage	Purpose
`$match`	Filter documents
`$group`	Group data (like GROUP BY)
`$sort`	Sort results
`$project`	Select fields
`$limit`	Limit results

Map-Reduce:

- Is a data processing paradigm used to condense large volumes of data into useful aggregated results.
- As of **MongoDB 5.0**, Map-Reduce is **deprecated** in favor of the Aggregation Pipeline.
- A Map-Reduce operation consists of **two** main JavaScript-based phases, followed by an optional final step.
    - **Map Phase**: Processes each input document and emits one or more key-value pairs.
    - **Reduce Phase**: Combines and condenses all values associated with the same key into a single result.
    - **Finalize (Optional)**: Performs final modifications to the output of the reduce phase.

Map-Reduce vs Aggregation

```
var mapFunction = function () {
    this.ratings.forEach(function(r) {
        emit(this.name, { total: r, count: 1 });
    }, this);
};

var reduceFunction = function (key, values) {
    var result = { total: 0, count: 0 };

    values.forEach(function(v) {
        result.total += v.total;
        result.count += v.count;
    });

    return result;
};

var finalizeFunction = function (key, reducedValue) {
    reducedValue.avg = reducedValue.total / reducedValue.count;
    return reducedValue;
};
```

```
db.products.aggregate([
 { $unwind: "$ratings" },
 {
  $group: {
   _id: "$name",
   total: { $sum: "$ratings" },
   count: { $sum: 1 }
  }
 },
 {
  $project: {
   _id: 1,
   value: {
    total: "$total",
    count: "$count",
    avg: { $round: [{ $divide: ["$total", "$count"] }, 2] }
   }
  }
 }
]);
```

OUTPUT

```
{
 "_id": "Laptop",
 "value": { total: 15, count: 4, avg: 3.75 }
}
{
 "_id": "Smartphone",
 "value": { total: 13, count: 3, avg: 4.33 }
}
{
 "_id": "Shoes",
 "value": { total: 7, count: 2, avg: 3.5 }
}
```

IMPORT / EXPORT

- **EXPORT:** It means Backup your data from MongoDB to an external file system.
  - `mongoexport --db shop --collection products --out products.json`
  -
    ```
    mongoexport --db shop --collection products \
    --fields name,price,category \
    --type=csv \
    --out products.csv
    ```

- **IMPORT:** It means inserting data from external file system (or any source) into MongoDB.
  - `mongoimport --db shop --collection products --file products.json`
  -
    ```
    mongoimport --db shop --collection products \
     --type csv \
     --headerline \
     --file products.csv
    ```

===THANK YOU===